

Design Reflection – Recycling Management System

Student: Maxamed Nuur **Date:** May 25, 2026

1. Architectural Decisions

Layered Architecture: Presentation → Application → Domain. Dependencies point inward. Domain has no UI or framework imports.

Strategy Pattern: Used for impact calculation. Interface `ImpactCalculationStrategy` with `SimpleImpactStrategy` and `WeightedImpactStrategy`. New strategies can be added without modifying existing code (OCP).

Constructor Injection: Dependencies passed via constructor, not created inside classes. Supports testing and DIP.

2. Design Principles Applied

Principle	Application
SRP	Product only manages data – impact calculation moved to Strategy
OCP	New strategies = new classes, no edits to existing code
DIP	Service depends on Strategy interface, not concrete classes

3. Testability

Domain layer tested with JUnit without UI. CI pipeline runs all tests on every push.

4. Technical Debt

- No persistence (in-memory only) – would add Repository pattern
- Limited error handling – would add custom exceptions

5. Conclusion

The project meets all requirements. The design is maintainable, testable, and follows SOLID principles.